INFORME NESTJS ARQUITECTURAS END POINT

Instructora: Yoly Alexandra Muñoz



Tecnólogo En Análisis Y Desarrollo De Software

Ficha:3407734

ELABORADO:

Andres Felipe Ceron Urrutia

CTPI POPAYAN

Cauca 2026

ACTIVIDAD DE APRENDIZAJE

Investigación de arquitecturas de software y construcción de un endpoint GET con NestJS y TypeScript

Programa de formación	Análisis y Desarrollo de Software (ADSO)
Tecnología	NestJS + TypeScript
Modalidad	Trabajo individual
Instructora	Yoly Alexandra
Aprendiz	Andres Felipe Ceron Urrutia

1. Situación problémica

Una organización requiere iniciar el desarrollo de un sistema de información que permita consultar datos desde una aplicación cliente. Antes de avanzar con funcionalidades más complejas, el equipo de desarrollo debe comprender las arquitecturas de software utilizadas en aplicaciones backend y construir un primer servicio web que responda correctamente a una solicitud HTTP GET.

Como aprendiz del programa ADSO, deberá investigar los conceptos solicitados, documentar la preparación del entorno de desarrollo y demostrar, mediante una aplicación funcional en NestJS y TypeScript, la implementación de un endpoint GET y el análisis de errores que pueden presentarse durante su construcción y ejecución.

2. Propósito de la actividad

Elaborar un documento técnico que evidencie el proceso de investigación, instalación, creación y ejecución de un proyecto NestJS con TypeScript; explicar diferentes tipos de arquitecturas de software y desarrollar un endpoint GET, identificando y documentando errores encontrados y sus respectivas soluciones.

3. Desarrollo de la actividad

Parte A. Investigación: arquitecturas de software

Investigue y explique, con sus propias palabras, los siguientes conceptos:

1. ¿Qué es una arquitectura de software y cuál es su propósito dentro del desarrollo de aplicaciones?

R/ La arquitectura de software es la estructura usada para el desarrollo de aplicaciones web/móvil de más alta categoría de un sistema informático, define las reglas y el uso que se va a tomar en el desarrollo web, que reglas utilizan, de qué forma se relacionan entre si los componentes principales y en resumen le da un orden a la forma y desarrollo de la App que se va a programar, lo que rige su diseño de construcción.

2. Arquitectura monolítica: definición, características, ventajas, desventajas y un ejemplo de uso.

R/ La arquitectura Monolítica es un diseño de modelo tradicional y de los más usados en el desarrollo de software, este consiste en que todo el desarrollo de la App se verá creado en la misma carpeta de proyecto por ejemplo, donde todo está relacionado entre si y se hará un solo despliegue, y todo estará guardado en un mismo repositorio.

Su código esta unificado es decir todo esta en un mismo lado, reside en un mismo repositorio, para editar alguna parte esta podría afectar el código debido a su alta cohesion y alto acomplamiento.

Sus ventajas serian la sencillez en el desarrollo de software debido a que esta arquitectura es de las mas fáciles de entender y manejar, su baja latencia debido a que todo esta tan unificado sus tiempos de respuestas entre componentes es corto, sencillez a la hora de encontrar errores y flujo de datos debido a la centralidad de este y tiene un despliegue rápido

Sus desventajas seria a la hora de el crecimiento del equipo, ya que como se maneja un código tan centralizado a la hora de programación el equipo podría entrar en conflicto debido a manipular tantos el mismo código siendo mas lento a la hroa de entrega, un error causaría un gran conflicto debido a que si no se encuentra como el código esta tan apegado este afectaría a los demás componentes dando una caída de la app y si se quisiera cambiar de lenguaje este seria mas complicado de hacer debido a lo centralizado del código.

Un ejemplo para este seríala creación de una App de venta de ropa por ejemplo, El catalogo, Carrito de compras, Pagos, Buscador y entre otros se desarrollaría junto se aplicaría que cuando se compre algo con los mismos componentes en alto acoplamiento el método de pago enviara a que cuando se compre se -1 existencia del stock del catálogo y mandaría la factura todo en el mismo servidor.

3. Arquitectura cliente-servidor: definición, responsabilidades del cliente y del servidor, ventajas, desventajas y un ejemplo.

R/ La arquitectura cliente-servidor es un modelo distribuido en el cual el que oferta los servicios o recursos se llama el servidor y el que los pide o demanda es el cliente dando a entender un ciclo de petición-respuesta.

La responsabilidad en un caso de ejemplo mayoritario seria de front-end como cliente y back-end como servidor funcionando de tal manera asi:

Front-end: Se encarga de mostrar visualmente los datos,como los cuadros de texto botones, relleno de formulario captación de clicks y mostrar un apartado visual agradable para el usuario.

Back-end: Este se encargara de la lógica de datos como por ejemplo hacer una suma, calcular un descuento o validar una acción como inicio de sesión, será el encargado también del manejo de los datos como su consulta, actualización, validación y almacenamiento de estos.

Las ventajas y desventajas de estos puede ser en este caso las ventajas, La simplicidad de este para equipos de baja gama debido a que solo usan el front-end como cliente ya que el servidor se encarga del trabajo duro, El manejo de datos y la comprensión de estos es simple debido a su centralización de control y tmb su facilidad de actualización debido a que es un sencillo servidor-cliente cliente-servidor si haces un cambio se vera reflejado rápidamente.

Las desventajas son el alto costo de mantenimiendo debido a que se requiere buena conexión a wifi y un servidor estable (pc) 24/7 para el control de los datos y la aglomeración de trafico en las respuestas debido a que si no se esta preparado estos pueden colapsar si se hacen varias peticiones al mismo tiempo.

Un ejemplo seria el revisar tus correos electrónicos donde el cliente/front-end se encarga de mostrar los correos que ya tienes y pedirte los datos para el ingreso/validacion o creación de uno existente o nuevo relativamente mientras que el servidor o back-end son los computadores de google en este caso que están conectados a un wi-fi 24/7 y son los encargados de recibir, validar y enviarte la comprobación de datos para una respuesta y que se muestre ya visualmente al final en el front-end.

4. Arquitectura de microservicios: definición, características, ventajas, desventajas y un ejemplo.

R/ La arquitectura de Microservicios es un diseño de modelo de desarrollo de software que consiste en dividir una aplicación en varios servicios independientes, donde cada uno se encarga de una función específica de la App. A diferencia de la arquitectura monolítica, aquí cada componente se desarrolla por separado, puede tener su propio repositorio y servidor, y se puede desplegar sin necesidad de modificar o detener toda la aplicación.

Su código está dividido en diferentes servicios, es decir, cada uno trabaja de manera independiente y se comunica con los demás mediante APIs o solicitudes. Esto permite que cada servicio tenga una menor dependencia de los otros, logrando un bajo acoplamiento y una mayor independencia a la hora de realizar modificaciones.

Sus ventajas serían la facilidad para trabajar en equipos de desarrollo más grandes, ya que cada equipo puede encargarse de un servicio diferente sin tener que modificar el mismo

código. También permite escalar únicamente los servicios que necesiten más recursos, facilita el mantenimiento y permite utilizar diferentes lenguajes de programación dependiendo de las necesidades de cada servicio. Además, si un servicio presenta un error, no necesariamente afectaría el funcionamiento de los demás.

Sus desventajas serían la mayor complejidad a la hora de desarrollar y administrar la aplicación, debido a que se tienen varios servicios independientes que deben comunicarse entre sí. También puede tener mayor latencia porque la información debe viajar entre diferentes servicios mediante solicitudes de red, su despliegue y configuración pueden ser más complicados y se necesita tener un mayor control de la comunicación y seguridad entre los servicios.

Un ejemplo para esta arquitectura sería la creación de una App de venta de ropa, donde el catálogo, carrito de compras, pagos, buscador y facturación se desarrollarían como servicios independientes. Por ejemplo, cuando un usuario compre una prenda, el servicio de pagos se encargaría de procesar el pago, luego se comunicaría con el servicio del catálogo para descontar una unidad del stock y finalmente el servicio de facturación generaría la factura. Cada uno trabajaría de manera independiente y se comunicaría con los demás cuando fuera necesario.

5. Arquitectura orientada a servicios (SOA): definición, características y diferencias frente a microservicios.

R/ La arquitectura Orientada a Servicios (SOA) es un modelo de diseño de software que consiste en dividir una aplicación en diferentes servicios que se encargan de realizar funciones específicas y que pueden ser utilizados por diferentes aplicaciones. Estos servicios se comunican entre sí para intercambiar información y trabajar en conjunto, permitiendo que diferentes sistemas puedan conectarse y compartir funcionalidades sin necesidad de estar desarrollados en el mismo proyecto.

Sus características serían que los servicios son reutilizables, es decir, pueden ser utilizados por diferentes aplicaciones sin tener que volver a desarrollar la misma función. También permite la comunicación entre diferentes sistemas mediante protocolos y servicios como SOAP o APIs, tiene una mayor independencia entre sus componentes y busca integrar diferentes aplicaciones para que puedan trabajar en conjunto. Además, suele utilizar un sistema centralizado llamado ESB (Bus de Servicios Empresariales), que permite organizar y controlar la comunicación entre los servicios.

Sus diferencias frente a los microservicios serían que SOA busca principalmente conectar e integrar diferentes sistemas y aplicaciones mediante servicios reutilizables, mientras que los microservicios buscan dividir una aplicación en pequeños servicios independientes, donde cada uno se encarga de una función específica. SOA suele manejar servicios más grandes y puede utilizar un sistema centralizado para comunicarlos, mientras que los microservicios buscan una comunicación más independiente y descentralizada. Además, en SOA los

servicios pueden compartir recursos o bases de datos, mientras que en microservicios se busca que cada servicio tenga sus propios datos y gestione su propia información.

Un ejemplo para esta arquitectura sería una empresa que tiene diferentes sistemas para ventas, inventario, facturación y clientes. Mediante SOA estos sistemas podrían conectarse entre sí utilizando servicios compartidos. Por ejemplo, cuando se realiza una venta, el sistema de ventas podría comunicarse con el servicio de inventario para consultar la existencia de un producto y luego con el servicio de facturación para generar la factura, permitiendo que todos los sistemas trabajen en conjunto sin necesidad de estar desarrollados dentro de una misma aplicación.

6. Arquitectura en capas: explicación de la separación por responsabilidades y ejemplo de sus capas.

R/ La arquitectura en capas es un modelo de diseño de software que consiste en dividir una aplicación en diferentes niveles o capas, donde cada una se encarga de una responsabilidad específica dentro del desarrollo de la App. Estas capas trabajan en conjunto, pero cada una tiene una función diferente, permitiendo que el código esté más organizado y sea más fácil de entender, modificar y mantener.

Su separación por responsabilidades consiste en que cada capa se encarga únicamente de realizar las tareas que le corresponden, evitando mezclar funciones diferentes en un mismo lugar. Por ejemplo, la capa de presentación se encarga de mostrar la información al usuario, la capa de lógica de negocio se encarga de procesar las operaciones y aplicar las reglas de la aplicación, y la capa de acceso a datos se encarga de comunicarse con la base de datos para guardar, consultar, modificar o eliminar información.

Sus ventajas serían la organización del código, debido a que cada parte tiene una responsabilidad específica, facilita encontrar y corregir errores, permite modificar una capa sin tener que cambiar necesariamente las demás y facilita el mantenimiento y el trabajo en equipo. Sus desventajas serían que puede aumentar la cantidad de código y la estructura del proyecto, puede generar dependencias entre las capas y, si no se organiza correctamente, una modificación en una capa podría afectar el funcionamiento de las demás.

Un ejemplo para esta arquitectura sería la creación de una App de venta de ropa, donde tendríamos tres capas principales:

- Capa de presentación: sería la interfaz que utiliza el usuario para buscar prendas, agregarlas al carrito y realizar compras.
- Capa de lógica de negocio: se encargaría de procesar las compras, calcular los precios, aplicar descuentos y verificar las reglas de compra.
- Capa de acceso a datos: se encargaría de consultar el catálogo, guardar las compras, actualizar el stock y almacenar la información de los usuarios en la base de datos.

De esta manera, cada capa tendría una responsabilidad diferente, pero todas trabajarían en conjunto para que la aplicación funcione correctamente.

7. Compare las arquitecturas investigadas e indique en qué tipo de proyecto sería conveniente utilizar cada una.

R/ Las arquitecturas investigadas tienen diferentes formas de organizar y desarrollar el software, cada una tiene sus propias características, ventajas y desventajas, por lo que dependiendo del tipo de proyecto que se quiera desarrollar será más conveniente utilizar una u otra.

La arquitectura Monolítica sería conveniente utilizarla en proyectos pequeños o medianos donde no se necesite una gran cantidad de desarrolladores ni una estructura demasiado compleja, debido a que todo el código se encuentra centralizado en un mismo proyecto. Por ejemplo, una App de inventario para una pequeña tienda donde se maneje el catálogo, las ventas y los clientes desde un mismo servidor.

La arquitectura Cliente-Servidor sería conveniente utilizarla en aplicaciones donde se necesite una comunicación entre un cliente que solicita información y un servidor que se encarga de procesarla y devolver una respuesta. Por ejemplo, una página web de correos electrónicos donde el usuario desde su navegador pueda consultar, enviar y recibir correos mientras el servidor se encarga de procesar y almacenar toda esa información.

La arquitectura de Microservicios sería conveniente utilizarla en proyectos grandes que necesiten crecer constantemente, manejar muchos usuarios y tener diferentes equipos de desarrollo trabajando al mismo tiempo. Por ejemplo, una plataforma de ventas en línea donde los pagos, el catálogo, los usuarios y la facturación funcionen como servicios independientes, permitiendo modificar o ampliar cada uno sin afectar directamente a los demás.

La arquitectura Orientada a Servicios (SOA) sería conveniente utilizarla en empresas grandes que necesitan conectar diferentes sistemas o aplicaciones que ya existen para que puedan compartir información y trabajar en conjunto. Por ejemplo, un banco que necesite conectar sus sistemas de clientes, cuentas, transferencias y facturación para que todos puedan comunicarse y compartir sus servicios.

La arquitectura En Capas sería conveniente utilizarla en proyectos que necesiten tener una buena organización del código y separar las responsabilidades de cada componente, facilitando su mantenimiento y modificación. Por ejemplo, una aplicación de gestión de estudiantes donde se tenga una capa de presentación para mostrar la información, una capa de lógica de negocio para procesar las matrículas y una capa de acceso a datos para guardar la información en la base de datos.

En conclusión, cada arquitectura tiene un propósito diferente y no existe una que sea mejor para todos los proyectos, ya que su elección dependerá del tamaño de la aplicación, la cantidad de usuarios, el equipo de desarrollo, la complejidad y las necesidades que tenga el software. Lo importante es elegir la arquitectura que permita desarrollar la aplicación de una manera organizada, eficiente y que facilite su crecimiento a futuro.

8. Explique qué tipo de arquitectura o estilo de organización reconoce en una aplicación backend desarrollada con NestJS. Justifique su respuesta.

R/ En una aplicación backend desarrollada con NestJS se puede reconocer principalmente una arquitectura en capas, debido a que su estructura permite organizar el código en diferentes partes dependiendo de la responsabilidad que tenga cada una. NestJS utiliza módulos, controladores y servicios para mantener una buena organización del proyecto y separar las funciones de cada componente.

Sus principales componentes serían:

- Módulos (Modules): Se encargan de organizar y agrupar los componentes relacionados con una misma funcionalidad, por ejemplo, usuarios, productos o ventas.
- Controladores (Controllers): Se encargan de recibir las peticiones del cliente, como consultar, crear, actualizar o eliminar información, y devolver una respuesta.
- Servicios (Services): Se encargan de manejar la lógica de negocio, es decir, procesar la información y realizar las operaciones necesarias.

La razón por la que se considera una arquitectura organizada por capas es porque cada componente tiene una responsabilidad diferente y trabaja en conjunto con los demás. Por ejemplo, cuando un usuario realiza una petición para consultar sus datos, el controlador recibe la solicitud, el servicio procesa la operación y posteriormente se pueden consultar los datos mediante una capa de acceso a datos o un repositorio.

En conclusión, NestJS permite desarrollar aplicaciones backend con una estructura modular y separación de responsabilidades, facilitando el mantenimiento, la organización y el crecimiento del proyecto.

Parte B. Instalación y creación del proyecto

Documente, mediante una secuencia ordenada, el procedimiento realizado para:

9. Verificar la instalación de Node.js y npm, incluyendo los comandos ejecutados y sus versiones.

R/ Se verifico correctamente la versión.

```
AndrezZ_7@DESKTOP-IVTSKF1 UCRT64 ~/Desktop
$ node -v
v24.21.0

AndrezZ_7@DESKTOP-IVTSKF1 UCRT64 ~/Desktop
$ npm -v
12.2.1
```

10. Instalar NestJS CLI y comprobar que quedó disponible en la terminal.

R/ Se verifico correctamente la instalación de npm y /cli.

```
AndrezZ_7@DESKTOP-IVTSKF1 UCRT64 ~/Desktop
$ npm install -g npm@12.2.1

changed 10 packages in 11s

AndrezZ_7@DESKTOP-IVTSKF1 UCRT64 ~/Desktop
$ npm install -g @nestjs/cli

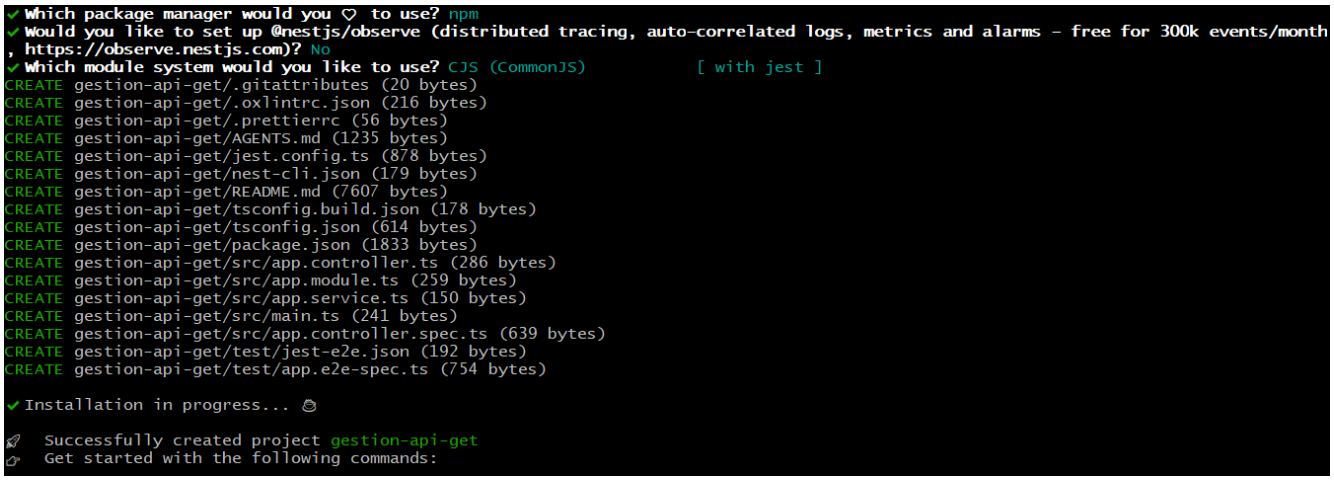
changed 91 packages in 10s
```

11. Crear un proyecto llamado gestion-api-get utilizando NestJS y TypeScript.

```
AndrezZ_7@DESKTOP-IVTSKF1 UCRT64 ~/Desktop
✨  We will scaffold your app in a few seconds..
```

12. Indicar las opciones seleccionadas durante el asistente de creación del proyecto.

R/ Se creo el proyecto y las opciones a dar son que tipo de package manager quería usar y como es un proyecto sencillo de prueba usamos npm (por mas seguridad se usaría pnpm) luego pregunto si quería usar el nestjs observe como no es necesario le di que no y en que modulo de sistema le puse que CJS with jest aunque no pregunto con que lenguaje por defecto usa type script.



13. Abrir el proyecto en Visual Studio Code e identificar los archivos y carpetas principales.

R/ Para abrir el proyecto en Visual Studio Code se puede utilizar la terminal ubicándose dentro de la carpeta del proyecto y ejecutando el comando code ., esto abrirá todo el proyecto para poder visualizar y editar su código.

Una vez abierto podemos identificar sus principales carpetas y archivos:

node_modules: contiene todas las dependencias y paquetes necesarios para el funcionamiento del proyecto.

src: contiene el código fuente principal de la aplicación, donde se encuentran los archivos encargados de su funcionamiento.

test: contiene los archivos destinados a realizar pruebas del proyecto.

package.json: contiene la información del proyecto, sus dependencias y los comandos que se pueden ejecutar.

package-lock.json: registra las versiones exactas de las dependencias instaladas.

nest-cli.json: contiene la configuración de NestJS CLI.

tsconfig.json: contiene la configuración del lenguaje TypeScript.

tsconfig.build.json: contiene la configuración utilizada para compilar el proyecto.

README.md: contiene información y documentación sobre el proyecto.

En conclusión, esta estructura permite mantener organizado el proyecto, separar los archivos dependiendo de su función y facilitar el desarrollo, mantenimiento y modificación de la aplicación desde Visual Studio Code.

14. Explicar la función de main.ts, app.module.ts, app.controller.ts y app.service.ts.

R/ Dentro de un proyecto desarrollado con NestJS existen cuatro archivos principales que se encargan de organizar y controlar el funcionamiento inicial de la aplicación. Cada uno tiene una responsabilidad diferente:

main.ts: Es el archivo principal de inicio de la aplicación, se encarga de ejecutar el servidor y poner en funcionamiento el proyecto. Es el punto de entrada donde NestJS comienza a levantar toda la aplicación.

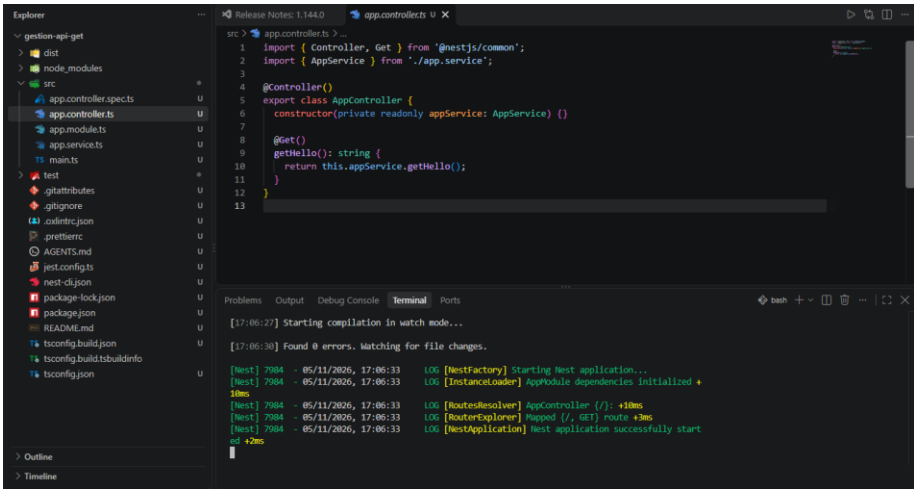
app.module.ts: Es el módulo principal de la aplicación, se encarga de organizar y registrar los diferentes componentes que se van a utilizar, como controladores y servicios. Es como el encargado de conectar y agrupar las partes principales del proyecto.

app.controller.ts: Se encarga de recibir las peticiones que llegan desde el cliente, por ejemplo, cuando alguien ingresa a una ruta o solicita información. Este recibe la petición y se comunica con el servicio correspondiente para obtener una respuesta.

app.service.ts: Se encarga de manejar la lógica de negocio, es decir, realizar los procesos y operaciones que necesita la aplicación. Por ejemplo, calcular valores, procesar información o devolver datos solicitados por el controlador.

En conclusión, estos cuatro archivos trabajan en conjunto para que la aplicación funcione correctamente. El main.ts inicia el servidor, el app.module.ts organiza los componentes, el app.controller.ts recibe las peticiones y el app.service.ts se encarga de procesar la información y devolver los resultados necesarios.

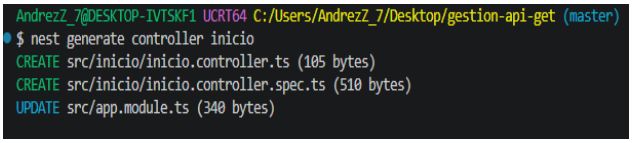
15. Ejecutar la aplicación en modo desarrollo y comprobar que el servidor inicia correctamente.

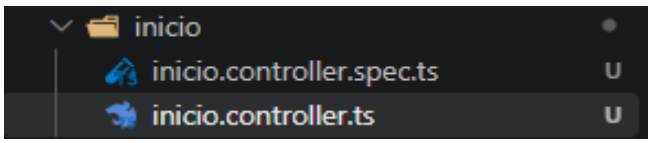


Parte C. Construcción de un endpoint GET

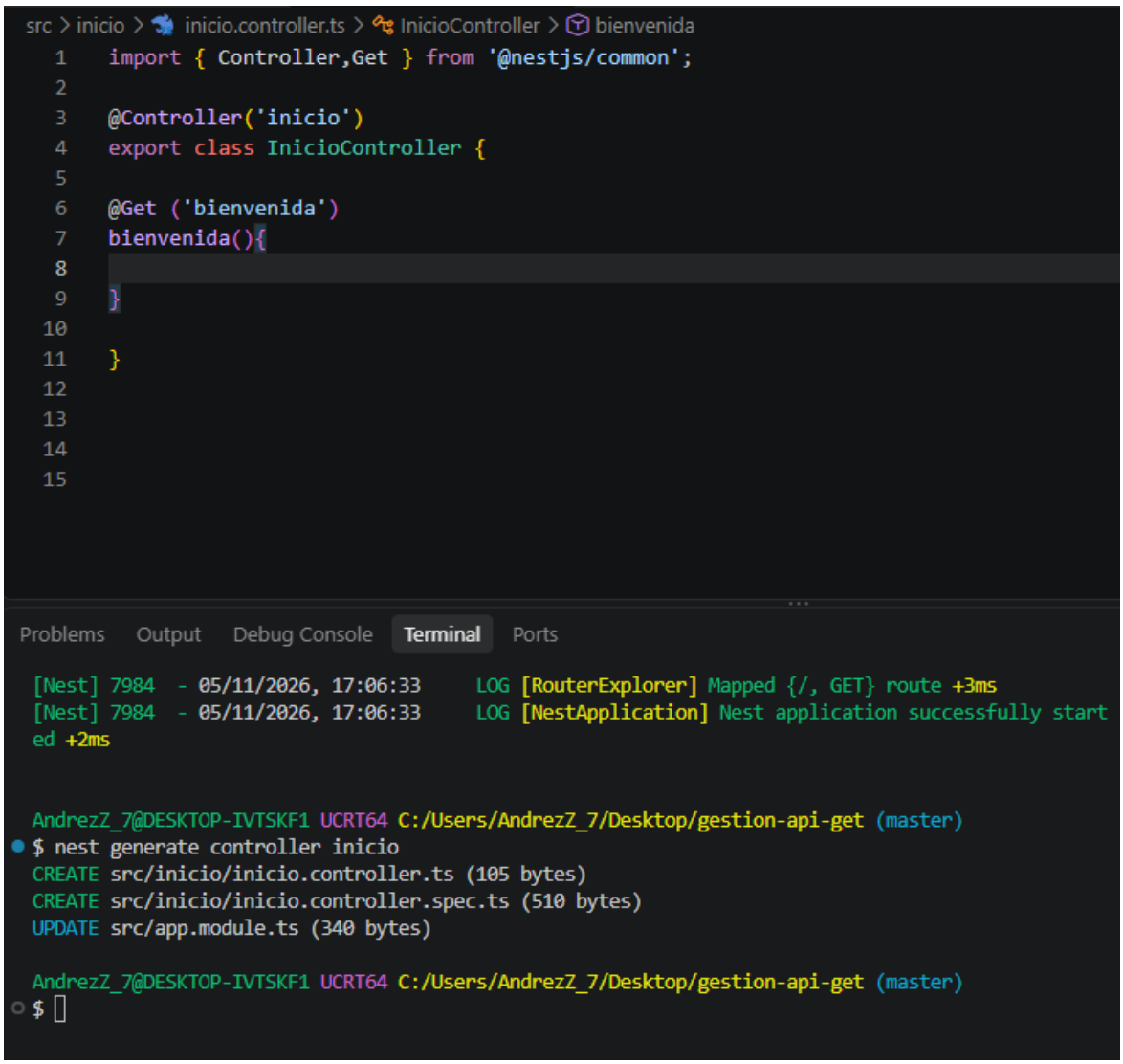
Sobre el proyecto creado, implemente un endpoint GET que responda con información de bienvenida y datos básicos de un sistema. El endpoint debe cumplir las siguientes condiciones:

16. Utilizar un controlador llamado InicioController o modificar el controlador principal.





17. Definir una ruta base /inicio y una ruta GET /bienvenida.

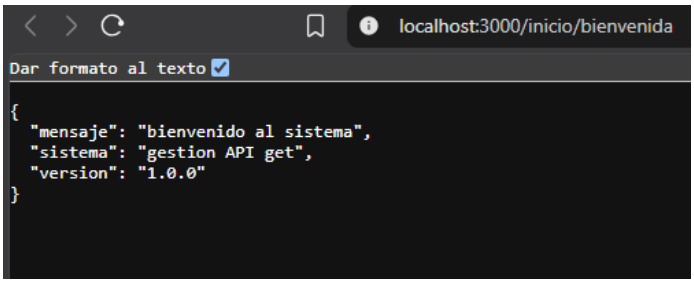


18. La URL completa que debe atender la solicitud es: http://localhost:3000/inicio/bienvenida.

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

19. Retornar una respuesta en formato JSON que incluya, como mínimo, un mensaje de bienvenida, el nombre del sistema y su versión.





20. Explicar la función de los decoradores @Controller() y @Get().

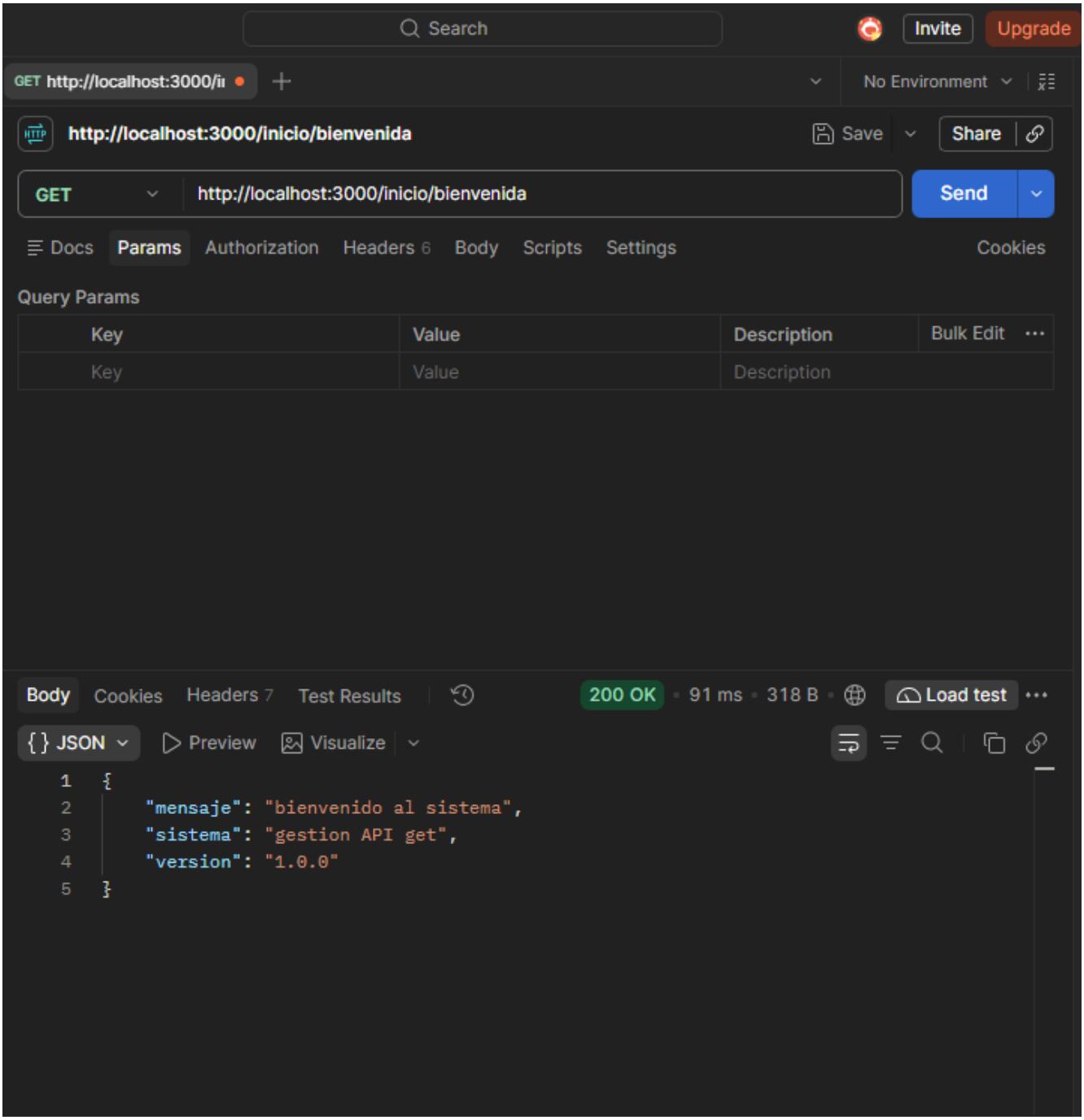
R/ El @ControllerInicio establece una ruta base y todas las que se creen dentro de el comenzaran con /inicio

Mientras tanto el @GetInicio indica el método que se ejecutara cuando alguien haga una petición http tipo get de la ruta bienvenida

21. Explicar la diferencia entre el nombre del método de TypeScript y la ruta HTTP que se configura mediante los decoradores.

R/ El nombre del método en TypeScript sirve para identificar la función dentro del código, mientras que la ruta HTTP se configura mediante los decoradores. Por ejemplo, el método puede llamarse bienvenida(), pero la dirección para acceder a él es /inicio/bienvenida. Ambos nombres pueden ser diferentes.

22. Probar el endpoint desde el navegador o Postman y registrar la respuesta obtenida.



Parte D. Identificación y análisis de errores

Durante la instalación, configuración, desarrollo y prueba del endpoint, analice los errores que se indican a continuación. Para cada caso, describa qué significa, una posible causa, cómo se identifica y cuál sería el procedimiento para solucionarlo. Si el error se presenta durante su práctica, incluya la evidencia real.

Error 404 – Cannot GET /getHello: Analice qué ocurre cuando se solicita una ruta que no ha sido declarada en los decoradores del controlador.

R/ Significa que se está intentando acceder a una ruta que NestJS no tiene registrada.

Error 404 – Cannot GET /inicio/bienvenida: Analice qué debe verificarse cuando la ruta aparentemente está bien escrita, pero NestJS no la reconoce.

R/ Puede ocurrir si el controlador no está registrado en el módulo principal o si la ruta está mal configurada. Se soluciona revisando app.module.ts y los decoradores.

Error de compilación de TypeScript: Explique cómo identificar errores de sintaxis, importaciones incorrectas o tipos incompatibles.

R/ Ocurre cuando hay errores en el código, como escribir mal una instrucción o importar algo incorrectamente. Se identifica en la terminal y se soluciona corrigiendo el código.

Error de instalación de dependencias con npm: Investigue posibles causas relacionadas con versiones de Node.js/npm, dependencias, caché o archivos de bloqueo.

R/ Puede ocurrir por problemas de internet, versiones incompatibles o dependencias dañadas. Se identifica por los mensajes de error de npm y se soluciona revisando las versiones y los archivos del proyecto.

Puerto ocupado: Explique qué sucede cuando el puerto 3000 está siendo utilizado por otro proceso y cómo resolverlo.

R/ Ocurre cuando otro programa está utilizando el puerto 3000. Se identifica con el error EADDRINUSE. Se soluciona cerrando el otro proceso o utilizando otro puerto.

Adicionalmente, registre al menos un error que haya ocurrido durante su propio proceso de trabajo. Incluya el mensaje exacto, una captura de pantalla, el análisis de la causa y la solución aplicada. Si alguno de los errores propuestos no se presentó, describa cómo podría reproducirse de forma controlada sin afectar otros proyectos.

R/ Durante el desarrollo del proyecto no tuve errores. La instalación, creación del controlador y prueba del endpoint funcionaron correctamente, por lo tanto, no cuento con capturas de errores reales.

4. Evidencias que debe contener el documento

- Portada con nombre del aprendiz, ficha, programa, instructor y fecha.
- Desarrollo de la investigación sobre arquitecturas de software, con fuentes consultadas.
- Capturas de la verificación de Node.js y npm.
- Captura de la instalación y verificación de NestJS CLI.
- Capturas del proceso de creación del proyecto y de su estructura en Visual Studio Code.
- Fragmentos de código del endpoint GET implementado.
- Captura de la terminal donde se evidencie que NestJS inició correctamente.
- Captura de Postman o del navegador mostrando la URL y la respuesta del endpoint.
- Tabla de errores con mensaje, causa, análisis y solución.
- Conclusiones personales sobre lo aprendido y referencias bibliográficas o webográficas.

5. Condiciones de entrega

- Entregar un único documento en formato Word o PDF, con títulos y secciones organizadas.
- El documento debe describir el procedimiento en el orden en que fue realizado.
- Las capturas deben ser legibles y corresponder al proyecto desarrollado por el aprendiz.
- El código debe estar acompañado de una explicación; no se aceptará únicamente la copia de fragmentos sin análisis.
- La investigación debe estar redactada con palabras propias y citar las fuentes consultadas.
- Adjuntar el proyecto NestJS completo o incluir el enlace al repositorio GitHub.
- El endpoint debe poder ejecutarse y probarse en el equipo del aprendiz.

6. Preguntas de reflexión

23. ¿Por qué es importante definir una arquitectura antes de iniciar el desarrollo de un sistema?

R/ Es importante definirlo antes de empezar el desarrollo de una app o sistema debido a que con una estructura definida indivudiualmente o grupalmente a la hora de la creación del proyecto este se va a tener un orden ya preestablecido el cual evitara conflictos y dependiendo según el modelo de app que se quiera hacer, la cantidad de desarrolladores o la conveniencia vinculada a que tipo de arquitetura se elija se optimizara mucho el desarrollo de esta para entregar un proyecto mejor y mas completo

24. ¿Qué ventajas ofrece NestJS para organizar una aplicación backend?

R/ NestJS permite organizar el código en módulos, controladores y servicios. Esto facilita el mantenimiento, la comprensión y el crecimiento de una aplicación backend.

25. ¿Qué diferencia existe entre un controlador y un servicio en NestJS?

R/ El controlador recibe las solicitudes y define las rutas, mientras que el servicio se encarga de realizar las operaciones y la lógica del programa.

26. ¿Por qué el nombre de una función no determina automáticamente la URL de un endpoint?

R/ Porque la URL se configura mediante los decoradores @Controller() y @Get(), mientras que el nombre de la función solo identifica el método dentro del código.

27. ¿Qué aprendió al identificar y solucionar errores durante la instalación y ejecución del proyecto?

R/ Primero que todo aprendi aparte de lo aprendido en clase mejor como se crea desde el gitbash o de una terminal el proyecto con los comandos de instalación npm install –g npm@12.2.1 y npm install –g @nestjs/cli para los paquetes y la creacion del proyecto con nest new xnombre que ya los tengo en mente para crear rapido cualquiero proyecto,aprendi a identificar la mayoria de los archivos importantes de el proyecto y a correrlo con npm run start:dev tmb el como crear controladores que se requiera con nest generate controller además de aprender la base para ya empezar a crear un proyecto o aplicación formalmente.

7. Resultado esperado

Un documento técnico que evidencie la investigación de los principales tipos de arquitecturas de software, el paso a paso de instalación y creación de un proyecto NestJS con TypeScript, la implementación y prueba de un endpoint GET, y el análisis de errores frecuentes y sus soluciones.